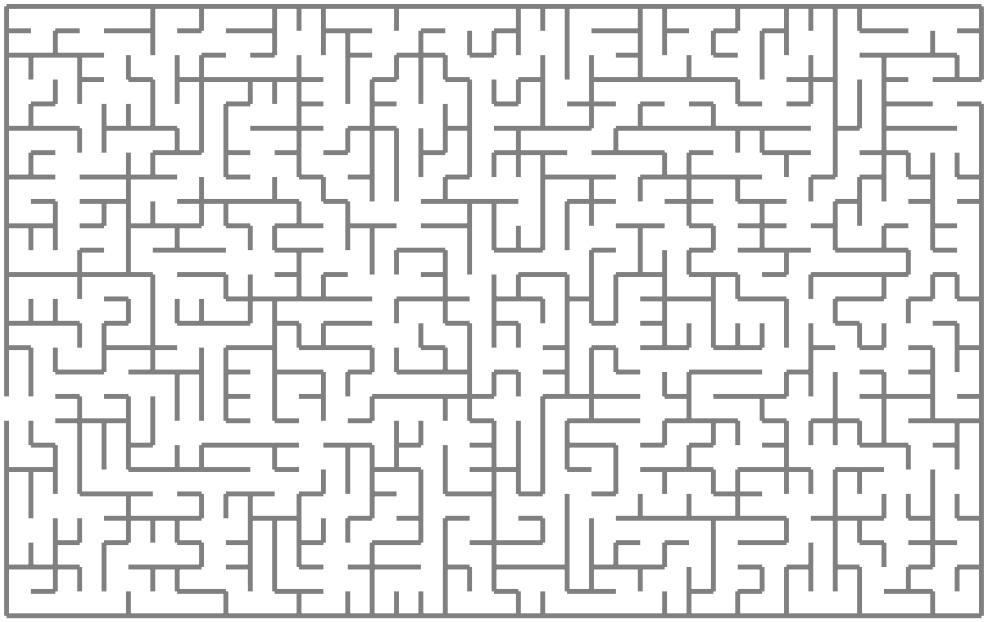
Higher order pattern unification using scope graphs

Version of February 10, 2026



Tudor Andrei

Higher order pattern unification using scope graphs

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Tudor Andrei
born in Bucharest, Romania

Programming Languages Group
Department of Software Technology
Faculty EEMCS, Delft University of Technology
Delft, the Netherlands
www.ewi.tudelft.nl

Cover picture: Random maze.

Higher order pattern unification using scope graphs

Author: Tudor Andrei
Student id: 5495822
Email: T.Andrei@student.tudelft.nl

Abstract

This work researches the possibility of using higher order unification in a typechecker that uses scope graphs. Scope graphs are a novel data structure for name resolution created by Statix. The Statix language is part of the language workbench Spoofax, created at TuDelft, and enables users to write a typechecker for their language. Out of the box, Statix can do first order unification between terms from the user's language which is enough to typecheck the majority of type system. However, dependently typed languages present a bigger challenge, which arises from type checking definitions with implicit arguments or pattern matching. We will propose a new unification algorithm for Statix, that is able to unify higher order terms (lambda functions) while using scope graphs for name resolution.

Thesis Committee:

Chair:	Prof. dr. C. Hair, Faculty EEMCS, TU Delft
Committee Member:	Dr. A. Bee, Faculty EEMCS, TU Delft
Committee Member:	Dr. C. Dee, Faculty EEMCS, TU Delft
University Supervisor:	Ir. E. Ef, Faculty EEMCS, TU Delft

Preface

Preface here.

Tudor Andrei
Delft, the Netherlands
February 10, 2026

Contents

Preface **iii**

Contents **v**

List of Figures **vii**

List of Tables **ix**

1 Introduction **1**
1.1 Contributions 1

2 Evaluation **3**

3 Related work **5**

4 Conclusion **7**

Acronyms **9**

A A **11**

List of Figures

List of Tables

Chapter 1

Introduction

Spoofax is a language workbench developed at TUDelft. Its main goal is to allow users to write end to end tooling for new languages in a declarative style and to offer abstractions for most common workflows. In Spoofax users will generally start with declaring the syntax of their language using a grammar file. This crucial step allows Spoofax to convert source code to an AST representation, which will be used as input to the other procedures in the framework.

The procedure which we are interested is typechecking, described through the Statix language. In many ways Statix is similar to λ-Prolog, a relational language, and dialect of Prolog which adds types and the ability to construct lambda terms. Akin to predicates in λ-Prolog, Statix has the concept of typed rules. Rules have multiple arguments (which can be a reference to a node in the scope graph or a term) and potentially a single output. Users are able to pattern match on the arguments.

1.1 Contributions

In this thesis the main contribution aims to improve the first order unification algorithm in Statix to one that uses higher order pattern unification (**RQ 1**). To validate the algorithm, we will create a simple dependently typed language in Spoofax.

Given the similarities between λ-Prolog and Statix, it is interesting to discuss the differences in expresivity and computability. λ-Prolog's syntax puts it closer to a theoretical specification of a type checker. It is very easy to mimic the rules of a typechecker with λ-Prolog by using predicates, universal/existential quantifiers and it's powerful higher order unification algorithm. On the other hand, Statix offers scope graphs as a name resolution algorithm instead of being able to generate fresh names through a universal quantifier. This makes it more practical for language builders as it easily supports mutual refferences, packages, imports and other types of non-lexical scoping. Therefore, we ask what are the differences between the two approaches when building a practical dependent type language (**RQ 2**).

Chapter 2

Evaluation

Evaluation here.

Chapter 3

Related work

Related work here.

Chapter 4

Conclusion

Conclusion here.

Acronyms

AST abstract syntax tree

DSL domain-specific language

Appendix A

A

Appendix here.